

Saran Citizen WhatsApp Bot: Detailed Implementation Plan

Phase 1: Sonpur Mela + District/Block Officer Directory

Version 1.1 · 23 Sep 2026 (adds 📍 Near me + on-site coordinate capture) · Branch `claude/whatsapp-n8n-llm-chatbot-o9gtyo`

Summary.

- **What:** a WhatsApp bot built on n8n where a citizen types "Hi" and chooses **Sonpur Mela** information or the **Officer Directory**.
- **Data:** it answers from Google Sheets that staff maintain, never from LLM guesses.
- **Near me:** a citizen can share their location to get the nearest police station, health centre or vet camp, with walking directions.
- **Growth:** new citizen services (Scheme Eligibility is next) plug in by adding a service row and a workflow, without rebuilding the bot.
- **Hosting:** an on-prem office machine behind a Cloudflare Tunnel.
- **Schedule:** go-live is targeted around **10 Nov 2026**, two weeks before the Mela opens, after three test-gated milestones.

0. Context

What exists today. The repo has a working prototype of a **Sonpur-Mela-only** bot:

- n8n + local Ollama + Postgres/pgvector
- CSV data loaded by `scripts/ingest.py`
- one monolithic workflow, `n8n/workflows/sonpur-mela-bot.json`, with keyword routing

The goal. Turn the prototype into a **citizen-services WhatsApp platform**. A citizen sends any message and gets a menu of services.

Phase 1 scope: exactly two services.

1. Sonpur Mela
2. District/Block Officer Directory

Scheme Eligibility is out of scope. It will be added later as a plug-in service, without rebuilding the core (see §14).

More services must plug in later without rebuilding the chatbot. Every interaction is logged for analytics and feedback.

Confirmed decisions

- **Build order:** Core platform → Mela → Directory → Analytics, with a test gate after each milestone.
- **LLM:** hosted Claude Haiku 4.5 (`claude-haiku-4-5`). Ollama and pgvector are retired.
- **Languages:** Hindi + English.
- **Hosting:** an **office machine, on-prem**, reached through a Cloudflare Tunnel (\$2).
- **WhatsApp:** the production number is live and verified.
- **Duty-staff mobile numbers:** approved for publishing.

- **Directory scope:** Saran district, sub-division, and block level, including **all block-level posts** (not only BDO/CO), plus police stations.
- **Data format:** the samples are indicative. The bot reads the recommended template format in §5b.
- **Data editing:** Google Sheets is where staff edit data. It is synced into Postgres every 10 min; the bot never reads Sheets live.

1. Senior-developer review findings

1a. Defects in the existing code (verified by reading it)

#	Finding	Evidence	Fix
E1	Only the first message of a webhook is processed	Extract Message reads <code>entry[0].changes[0].messages[0]</code> . Meta can batch several entries/messages into one POST	Router splits all <code>entry[] × changes[] × messages[]</code> into separate items
E2	No webhook signature check: anyone who knows the URL can POST fake messages, spend LLM credits and spam replies	No X-Hub-Signature-256 check anywhere	Verify HMAC-SHA256 of the raw body with <code>META_APP_SECRET</code> ; reject on mismatch
E3	Graph API pinned to <code>v19.0</code>	Send WhatsApp Reply URL	[Likely] v19 is at or near Meta's ~2-year deprecation. Make it <code>GRAPH_API_VERSION</code> in env and set it to the current version at build time
E4	<code>N8N_BASIC_AUTH_*</code> in compose	<code>docker-compose.yml</code>	[Likely] Removed in n8n 1.x, which uses built-in user management. The editor is effectively protected only by the owner account and is exposed on 0.0.0.0:5678 . Fix: bind to 127.0.0.1, expose only <code>/webhook/*</code> through the Cloudflare Tunnel, and reach the editor over Tailscale
E5	<code>n8nio/n8n:latest</code> , <code>ollama:latest</code>	compose	Pin exact versions. A silent n8n upgrade mid-Mela can break node schemas
E6	No <code>N8N_ENCRYPTION_KEY</code> set	compose	If the n8n volume is lost, all stored credentials become unreadable. Set it explicitly and back it up
E7	Workflow JSON has no fixed <code>id</code>	<code>id: None</code> in the export	Required for registry dispatch (see R1)
E8	Non-text messages (voice notes, images) get a silent ack	Ack Non-Text	Voice notes will be common. Reply in Hindi: "Please type or use the menu" and log the type (voice-to-text is a candidate future service)

1b. Design risks fixed in this plan

#	Risk	Fix
R1	n8n assigns new workflow IDs on import, which breaks dispatch-by-ID	Every workflow JSON carries a fixed <code>id</code> . Deploy with n8n <code>import:workflow</code> via <code>scripts/n8n_import.sh</code> . UI edits are followed by <code>scripts/n8n_export.sh</code> and a commit

#	Risk	Fix
R2	Meta status callbacks (sent/delivered/read) run about 3× the inbound volume	Dropped at the first node. <code>EXECUTIONS_DATA_SAVE_ON_SUCCESS=none</code> , prune >72h. <code>core.message_log</code> is the audit trail
R3	Stale buttons tapped after the session expired	Self-describing IDs <code>svc:action:arg</code> (e.g. <code>mela:fac:medical</code>). Routing by ID prefix works without session state
R4	WhatsApp field limits: button title ≤20, list row title ≤24, row description ≤72, list button ≤20, ≤3 buttons, ≤10 rows, interactive body ≤1024, text ≤4096	Meta rejects the whole message if any one field is over. The send workflow validates and truncates, and CI <code>check_templates.py</code> fails on overflow in hi or en
R5	Admin alerts and the daily report are business-initiated, so they are blocked outside the 24h window	An approved utility template <code>admin_alert</code> (1 variable), plus email as backup
R6	Non-atomic sync can expose half-loaded tables	One SQL function per table does delete+insert in a single transaction
R7	The in-charge roster would need facilities × 32 days × 3 shifts ≈ thousands of rows	Confirmed by the sample: the roster is a standing 3-shift pattern (the same person every day). <code>duty</code> has no date; a blank shift means all-day
R8	Concurrent messages from one user race on session state	<code>sessions.version</code> optimistic lock, with one retry
R9	Sub-workflows have no HTTP entry, so they can't be tested	<code>core/99-test-harness</code> webhook, active only when <code>ENV=test</code>
R10	Load tests would hit the real Claude API and Meta	<code>ANTHROPIC_BASE_URL</code> and <code>WA_API_BASE</code> point at local mocks in test mode
R11	Prompt caching won't help	Haiku 4.5's minimum cacheable prefix is 4096 tokens and <code>history.md</code> is smaller. No caching; cost is negligible
R12	A language-picker first step causes drop-off	Auto-detect (Devanagari → hi, otherwise en). Every menu has a "🌐 English/हिंदी" toggle
R13	Sending citizen text to a foreign API needs departmental approval	Get written sign-off in week 1. <code>LLM_ENABLED=false</code> gives a fully working menu-only bot. The phone number is never sent to the LLM
R14	Spam/abuse, or a loop with another bot	Per-user rate limit: >20 inbound per 5 min → one "please slow down" reply, then ignore for 10 min
R15	No error path: a failed execution means the citizen gets silence	An n8n Error Trigger workflow sends the citizen a "Sorry, please try again / menu" reply and an <code>admin_alert</code> to staff
R16	No monitoring	An external uptime check on <code>/healthz</code> (tunnel route to an n8n health webhook, checked by UptimeRobot every 5 min with SMS/email alerts), a daily sync-freshness check, and disk-usage alerts
R17	A "zero core changes" gate is too absolute	Gate = no core <i>feature</i> changes to add a service. Core bug fixes go in separate commits with tests

2. Architecture (on-prem office machine, as decided)

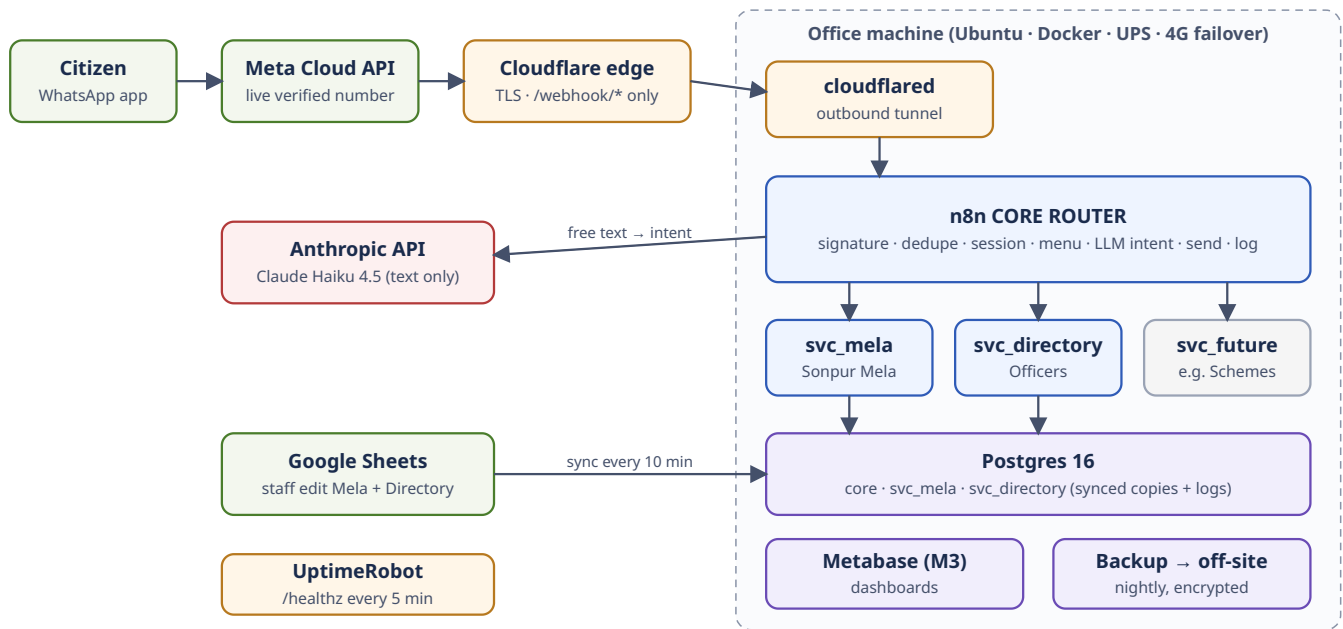


Figure 1. Phase 1 architecture: the core router plus plug-in services, hosted on-prem behind a Cloudflare Tunnel.

2.1 Why Cloudflare Tunnel instead of port-forwarding/Caddy

[Certain] Meta requires a public HTTPS URL with a valid certificate. An office machine usually has no static IP, sits behind NAT, and has an IT firewall.

- **cloudflared connects outbound only**, so there's no static IP, no port forwarding, and no inbound firewall rule. It gets a real TLS certificate on `bot.<domain>`, and Cloudflare Access rules block every path except `/webhook/*` and `/healthz`.
- **Cost**: free tier. It needs a domain (or subdomain) whose DNS is on Cloudflare.
- **Fallback**: if IT forbids Cloudflare, use port-forward 443 + Caddy + a static IP from the ISP (≈₹500–1,500/month extra). The compose file keeps this as a `caddy` profile.

2.2 Office-machine requirements (on-prem is now the #1 availability risk)

Item	Minimum	Why
CPU / RAM / disk	4 cores, 16 GB RAM , 256 GB SSD	n8n + Postgres + Metabase + OS headroom (no local LLM)
OS	Ubuntu 24.04 LTS (dedicated, not someone's daily PC)	Docker restart policies; no Windows updates rebooting it mid-Mela
Power	Online UPS ≥30 min + building inverter/genset	Mela month in Nov–Dec; power cuts are likely [Likely]
Internet	Primary broadband + 4G/5G dongle or router failover	Meta retries for a limited time only; long outages mean lost replies
Boot	BIOS "power on after AC loss"; Docker + cloudflared as systemd services	A reboot must recover by itself with no one on site
Remote access	Tailscale on the machine and admin laptops	Night-time fixes without visiting the office
Physical	Locked room, labelled "do not switch off"	The most common real-world on-prem outage cause [Likely]

2.3 Disaster recovery (cold standby)

- **Nightly backup:** `pg_dump` + the env/keys file, encrypted, copied off-site.
- `docs/runbook.md` → "Move to cloud in 1 hour":
 1. Rent any VPS.
 2. `git clone`.
 3. Restore the dump.
 4. Install `cloudflared` with the same tunnel token.

The DNS and the Meta webhook URL stay the same, so citizens notice nothing. **This is rehearsed once before go-live** (M2 gate).

Component versions (pinned in compose; exact tags chosen at M0 start): `n8n 1.x` · `postgres:16` · `cloudflare/cloudflared` · `metabase (M3)`.

Environments. Two compose projects run on the same machine:

- **staging:** Meta test number, `staging.<domain>` tunnel route, `ENV=staging`.
- **prod:** the live number, `bot.<domain>`, `ENV=prod`.

Each has its own DB and env file. Promotion from staging to prod imports the same workflow JSON files.

3. Core platform design

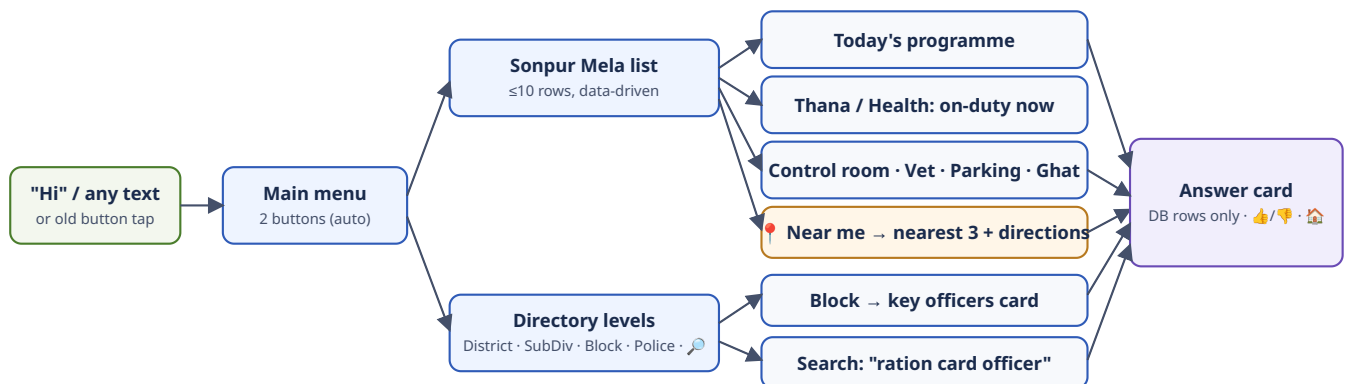


Figure 2. Citizen journey. Every answer comes from verified sheet data, never from LLM text.

3.1 Router pipeline (`core/00-router`, node by node)

1. **Webhook POST** `/webhook/wa`. Raw body enabled; responds **200 immediately** (Meta retries on a slow ack).
2. **Verify signature** (Code; `NODE_FUNCTION_ALLOW_BUILTIN=crypto`). On mismatch: log a security event and stop.
3. **Explode + filter** (Code). Walks all entries, changes, and messages. Drops statuses. Output: one item per message.
4. **Begin turn** (Postgres: `core.begin_turn(wa_id, msg_id, ts, type)`), a single round trip that:
 - HMACs the phone number with `PHONE_HASH_SECRET`
 - dedupes (`INSERT ... ON CONFLICT DO NOTHING`)
 - upserts the citizen (the number is stored `pgp_sym_encrypt`-ed)
 - checks the rate limit
 - loads the session and its version
 - returns `{is_dup, rate_limited, wa_hash, lang, state, context, version, first_contact}`
5. **Stop** if `is_dup`. If `rate_limited`, send one notice and stop.

6. **Normalize input** (Code). Produces `kind` (text/button/list/location/unsupported), `id`, `text`, `lat/lon`, and the detected language.
 7. **Resolve route** (Code), in priority order:
 - a global command (`hi|hello|namaste|menu|0|start|मेनू|help`) → core menu
 - a `lang:* id` → switch language
 - an `fb:* id` → feedback
 - an `id` prefix matching a registry `id_prefix` → that service
 - a session state prefix → that service
 - free text → LLM classify
 - otherwise → menu
 8. **LLM classify** (`core/02-llm`, only for free text with no active flow expecting text). A confident result becomes a service route with a synthetic `input.intent`. Low confidence or an error sends the menu and logs to `unanswered`.
 9. **Dispatch** (Switch → Execute Workflow; the workflow ID comes from the registry row, "wait for completion" on).
 10. **Validate output** (Code). Checks the contract schema. Invalid output → error path.
 11. **Decorate** (Code). On the first contact, the privacy notice is prepended. If `done=true`, the feedback buttons `[👍][👎][🏠 Menu]` are appended.
 12. **Send** (`core/01-send`, per message):
 - render the hi/en template
 - validate limits (R4)
 - **phone guard**: every phone-like token must exist in `core.allowed_numbers` (a view: union of service phones + 112/108/1098)
 - POST to Graph API
 - log the outbound message
 - retry once on 5xx/429
 13. **End turn** (Postgres: `core.end_turn(wa_hash, version, next_state, context, log)`). On a version conflict, re-run once from step 6.
- **Error Trigger** (`core/08-error`). Sends the apology + menu to the citizen and an `admin_alert` with the execution ID.

3.2 Service contract

```

INPUT  { wa_hash, lang: "hi"|"en", state, context, now_ist, is_admin,
         input: { kind, id, text, lat, lon, intent?: {subtype, slots} } }
OUTPUT { messages: [ {type:"text", body} |
                     {type:"buttons", body, buttons:[{id,title}] (≤3)} |
                     {type:"list", body, button, sections:[{title, rows:[{id,title,description}]}] (≤10
rows)}} |
         {type:"location", lat, lon, name, address} |           // map pin, opens in Maps
         {type:"location_request", body} ],                     // "Send location" button
         next_state, context, done, log: { subtype, resolved } }
```

Rules:

- Services never call Meta or write to `core.*`.
- Every state and every ID starts with the service's `id_prefix`.
- A service must handle any stale ID of its own gracefully.

3.3 Registry (`core.services`)

`service_key` PK, `id_prefix` UNIQUE, `title_hi`, `title_en` (≤20), `description_hi/en` (≤72), `menu_order`, `enabled`, `workflow_id`, `intent_hint_en`, `intent_hint_hi`, `subtypes` jsonb {key: hint}, `keywords` text[].

- **Main menu.** ≤ 3 enabled services → reply buttons. Otherwise a list, where the row description comes from `description_*`.
- **LLM prompt.** Built from `services` + `subtypes` at call time, so new services are classified automatically.

3.4 LLM specification (`core/02-llm`, HTTP Request → `POST /v1/messages`)

Classifier

- **Request:** `claude-haiku-4-5`, `max_tokens 300`, `temperature 0`, `timeout 8s`. The system prompt is generated from the registry.
- **Output:** `output_config.format` JSON schema: `{service_key: enum[...enabled keys, "none"], subtype: string, slots: {block, department, date, facility_type}, lang: enum[hi,en], confidence: number}`
- **Threshold:** accept if `confidence ≥ 0.6` and `service_key ≠ none`.

Mela Q&A (used by `svc_mela` via the same sub-workflow with `mode=qa`)

- **System prompt:** `history.md` + do's & don'ts, loaded from the DB (`svc_mela.qa_context`).
- **Output:** `{answerable: bool, answer: string ≤600 chars}`, in the user's language, with no phone numbers.
- **Fallback:** if `answerable=false`, the bot says it doesn't know and shows the Mela menu.

General

- **Logging:** every call records `llm_tokens` and latency.
- **Kill switch:** with `LLM_ENABLED=false`, both calls are skipped.

3.5 Conversation rules

- **Session timeout:** 30 min idle. The next message starts at the menu, but self-describing IDs still work.
- **Menu shortcuts:** "0" or "menu" works anywhere. Each result ends with the feedback + menu buttons.
- **Unsupported message types** (voice, image, video, sticker, document): the bot replies with a template and shows the menu.
- **Location messages:** routed to the active service if it declares `accepts_location`. Otherwise they go to the service with a pending location request, and if there is none, to `svc_mela`'s "nearest" handler during the Mela window (§6.1a) or to the menu outside it.
- **Admins:** `core.admins` (hashed numbers, seeded from `ADMIN_WA_NUMBERS`) sets `is_admin` in the service input. Admin-only IDs (`*:adm:*`) are rejected for everyone else.

4. Data model (`sql/*.sql`, applied in order; idempotent)

core schema (`00_core.sql`)

- `services` as in 3.3.
- `templates` (key PK, hi, en, max_len).
- `citizens` (wa_hash PK, wa_number_enc bytea, lang, first_seen, last_seen, notice_shown_at, msg_count).
- `sessions` (wa_hash PK, state, context jsonb, lang, version int, updated_at).
- `message_log` (id bigserial, wa_msg_id text UNIQUE NULL, wa_hash, direction in/out, service_key, state, subtype, kind, text, via button|list|text|llm|system, resolved, llm_tokens, latency_ms, error, ts).
 - Indexes: (ts) and (service_key, ts).
- `feedback` (id, wa_hash, message_log_id, rating smallint, ts).
- `unanswered` (id, wa_hash, text, service_key, reason, ts).
- `sync_runs` (id, service_key, source, status ok|rejected|error, rows, errors jsonb, ts).
- `rate_limit`: handled inside `begin_turn` with a count over `message_log` in the last 5 min (indexed), so no separate table.
- **Functions:** `begin_turn`, `end_turn`, `purge_older_than(days)`.

- **View** `allowed_numbers`, plus analytics views `v_daily`, `v_top_subtypes`, `v_feedback`. These expose no raw numbers.

`svc_mela` schema (`10_svc_mela.sql`), shaped by the real 2025 sample

- **shifts** (`shift_no` 1..3, `label`, `start_time`, `end_time`).
 - Seeded 06:00–14:00, 14:00–22:00, and 22:00–06:00; this pattern is identical in the Control Room and Thana tabs.
 - `current_shift(now_ist)` handles Shift-3 crossing midnight.
- **places** (`id`, `category`, `name_en`, `name_hi` NULL, `lat` NULL, `lon` NULL, `sort`).
 - `lat/lon` come from the sheet.
- **place_coords** (`place_id` PK, `lat`, `lon`, `captured_by_hash`, `captured_at`): coordinates captured on-site by admins (§6.1b).
 - **The sync never touches this table.**
 - Effective location = `COALESCE(sheet lat/lon, captured lat/lon)`, so a value typed into the sheet overrides a capture.
 - The view `v_places_geo` exposes it.
 - `category` is one of: `health_centre`, `thana`, `vet_camp`, `parking`, `ghat`, `accommodation`, `other`.
- **duty** (`place_id`, `shift_no` NULL, `person_name`, `phones` text[], `role` NULL).
 - `shift_no` NULL means all-day. Vet camps have no shifts. Merged cells in the source mean the same person on every shift.
 - `phones` is an array, because source cells hold "8709898293 & 7970663565".
- **control_room_desks** (`desk_en`, `desk_hi`, `sort`) and **control_room_duty** (`desk`, `shift_no` NULL, `person_name`, `designation`, `phones` text[]).
 - The source is a desk × shift grid: Magistrate, Police, Sanitation & Water, Electricity, Health.
- **control_room_public** (`label`, `phone`): the public helpline number(s). **Missing from the sample** (see §5a).
- **events** (`date`, `department`, `programme_en`, `programme_hi` NULL, `time` NULL, `venue` NULL).
 - The sample has only date, department, and artist/programme.
- **guidelines** (`kind` do|dont, `text_hi`, `text_en`, `sort`). **Not in the sample.** The menu row is hidden until rows exist.
- **qa_context** (`id`=1, `content`): history.md + guidelines.
- **Functions:** a `replace_<table>(jsonb)` function per table, `current_shift()`, and `on_duty(category, now_ist)`.
- **nearest(category, lat, lon, n=3)**: haversine distance over `v_places_geo`, limited to `active` AND `verified` AND coordinates present.
 - Returns `place`, `distance_m`, on-duty person, phone.
 - Plain SQL is enough for about 50 sites, so no PostGIS is needed.

`svc_directory` schema (`20_svc_directory.sql`), shaped by the real sample

- **officers** (`id`, `level`, `unit_en`, `unit_hi` NULL, `designation_en`, `designation_hi` NULL, `person_name`, `phone` NULL, `email` NULL, `keywords` text[], `sort`, `last_verified`).
 - `level` is one of: `district`, `subdivision`, `block`, `circle`, `police`.
 - `unit_en` holds the block, circle, or thana name ("Amnour"), the subdivision ("Sonpur"), or "Saran" for district level.
- **blocks** (`name_en`, `name_hi`, `aliases` text[]): the 20 Saran blocks. Aliases include Sadar = Chapra, Rivilganj = Revilganj, and Isuapur = Ishuapur.
 - Used for the paginated list and fuzzy matching.
- **Indexes:** trigram indexes on `unit_en`, `designation_en`, `person_name`, and `blocks.aliases`.
- **Function:** `replace_officers(jsonb)`.

5. Data: what the samples show, and the sheet format the bot will read

5a. Findings from the samples (Sonpur_Mela_Data.xlsx + Directory Google Sheet)

#	Finding	Evidence	Consequence / action
D1	Both files are print layouts, not data tables. They use merged title rows, two-row headers, several sections stacked in one tab (the directory has BDO, CO, Thana, District, and Sub-division stacked), and shifts as column groups	Every tab	A sync can't reliably parse these. Decision: we convert them once into a "bot format" Google Sheet: one tab per table, one header row, no merged cells. Staff keep editing that sheet (§5b). The one-time conversion script <code>scripts/convert_samples.py</code> is part of M1/M2
D2	Legacy Hindi font encoding	Health centre "Nakhas ¼u[kk'k½" is Kruti Dev text, not Unicode, so it would reach citizens as garbage	The sync rejects cells containing Kruti Dev markers (¼ ½ [k patterns). Hindi must be typed in Unicode. The converter strips the part and flags it
D3	Invalid phone	Thana "Sawaich Ghat" Shift-2: 709095094 (9 digits)	Validator: mobiles must be 10 digits starting 6–9; landlines need an STD code with a leading 0. The row is rejected with its row number in the alert
D4	A landline stored without its leading 0 looks like a mobile	Forest Division Officer 6152232660 (Chapra STD 06152). DM 06152-240001 is correct	Excel stores numbers as integers and drops the leading 0. The phone column must be plain text in the sheet. 6152.../6158... patterns are flagged for manual check
D5	Multiple people/phones in one cell	Control Room Sanitation & Electricity desks hold 2 officers and "8709898293 & 7970663565"	<code>phones</code> is stored as an array. The bot format has separate <code>phone_1</code> and <code>phone_2</code> columns
D6	Vertical merged cells mean "same for all shifts"	Control Room G4:G6, H4:H6, I4:I6	Converter: the value applies to shifts 1–3. Bot format: a blank shift means all-day
D7	Suspected copy-paste errors	Director DRDA and Incharge Legal Section share 9031071905 . Nagar PS and Nagra PS share an email. "Bahadur Prasad Yadav" is on duty at two thanas with different numbers	A sync warning (not a reject) lists duplicate phones/emails across different people; the data owner confirms
D8	The Control Room tab lists internal duty officers but no public helpline number	The tab has no "call this number" line	Citizens need one number. Ask the district for the public control room number; it goes in <code>control_room_public</code> . The desk roster is shown underneath
D9	Accommodation and do's & don'ts are not in the sample; Ghats are (6)	Sheet list	Mela menu rows are data-driven : a category with 0 rows is hidden automatically. Ghats become a menu row
D10	No lat/lon for any site	Blank columns	The "📍 Near me " feature (§6.1a) depends entirely on coordinates . The temporary sites only exist once they are set up, so coordinates are captured on-site by staff through the bot's admin tool (§6.1b) in the set-up week. Until then the feature switches itself off per category

#	Finding	Evidence	Consequence / action
D11	Schedule is 2025 (22 Nov–7 Dec 2025, with gaps on 4 and 6 Dec). No time or venue. Artist names are comma-joined	Mela Schedule tab	Good staging seed. The sync warns if no event date falls in the next 30 days. "Today" with no row → "No programme listed today" + the next listed date
D12	Police thanas aren't linked to blocks (39 district thanas vs 20 blocks). Sub-division rows all read "Subdivisional Officer" and only the email reveals Chapra/Marhaura/Sonpur	Directory sheet	A "block → department" flow doesn't fit. The directory is navigated by office level first (§6.2). Converter sets <code>unit_en</code> for sub-divisions from the email
D13	All data is English only	Both files	The bot's own text is in hi/en. Names and places stay in English unless staff fill the optional <code>*_hi</code> columns. We won't machine-transliterate personal names , because wrong Hindi names on an official channel are worse than English ones
D14	Long names exceed WhatsApp list limits	"Veterinary Hospital Mobile Ambulance Van, Dist. Animal Husbandry Office" (74 chars, the list-row title limit is 24)	Places are listed in text cards, not list rows . Lists are used only for categories, blocks, and pages
D15	Mela duty staff numbers are personal mobiles (doctors, SIs)	Health/Thana tabs	Decided: publishing is approved. Keep a copy of the approval order in <code>docs/approvals/</code> . A <code>settings.publish_duty_phones</code> flag (default TRUE) allows an instant switch to names + helpline if an officer objects
D16	Police display names carry disambiguation suffixes ("Randhir Kumar-2", "Mukesh Kumar-01")	Directory Thana section	The display strips a trailing <code>- \d+</code> ; the stored value is kept as-is

5b. Recommended data format (the samples may change, so this is the target format)

Design rule: keep the sheet **as close as possible to how staff already write the duty orders**, one row per place with shift columns. Remove only the things that break machines: merged cells, two-row headers, several tables stacked in one tab, numbers stored as numbers, and legacy fonts. The sync (not the staff) turns this into normalised DB tables. Foreign-key tabs (a separate `duty` tab pointing at `place_id`) were rejected because staff mis-type IDs.

Ten rules for every tab (printed on a frozen `README` tab in each spreadsheet):

- Row 1 holds the headers exactly as given. Never rename, merge, or reorder them; adding columns at the end is fine.
- One table per tab. There are no title rows and no blank rows in the middle.
- Phone columns are **Plain text** format. Use a 10-digit mobile (9431012345), or a landline with its STD code and a leading 0 (06152-240001). There is one number per cell; a second number goes in `phone_2`.
- Dates are `YYYY-MM-DD` and times are 24-hour `HH:MM`.
- Hindi is typed in **Unicode** (Google Input Tools or the Windows Hindi keyboard), **never Kruti Dev**.
- Dropdown columns accept only the listed values.
- `active` is TRUE/FALSE. Set it to FALSE to hide a row instead of deleting it; this keeps history.
- `verified` is ticked only after the number has been called or checked against the order. Unticked rows never reach citizens in production.
- Each tab has an `updated_by` column for the staff member's name.
- The `id` column is a short unique code (e.g. `TH01`, `HC03`) and must never be reused.

Spreadsheet 1: "Sonpur Mela – Bot Data"

Tab	Columns (header row, exactly)	One row =
README	the rules above + a contact for the data owner	n/a
settings	key, value → mela_start, mela_end, shift1_start 06:00, shift2_start 14:00, shift3_start 22:00, public_helpline_1, public_helpline_2, mela_center_lat, mela_center_lon, mela_radius_km (default 5)	one setting
places	id, category▼, name_en, name_hi, location_en, location_hi, lat, lon, hours, s1_name, s1_phone, s2_name, s2_phone, s3_name, s3_phone, allday_name, allday_phone, phone_2, notes, active, verified, updated_by	one thana / health centre / vet camp / parking / ghat / accommodation / help desk. Shift columns are blank where not applicable (parking, ghat)
control_room	desk▼ (Magistrate, Police, Sanitation & Water, Electricity, Health, other), s1_name, s1_designation, s1_phone, s2_..., s3_..., allday_name, allday_designation, allday_phone, phone_2, active, verified, updated_by	one desk (the sample's grid maps 1:1)
events	id, date, time, programme_en, programme_hi, artists, department, venue, is_highlight, active, verified, updated_by	one programme item (several per day allowed)
guidelines	id, kind▼ (do, dont, emergency), text_en, text_hi, sort, active, verified, updated_by	one instruction
faq (optional)	id, question_en, question_hi, answer_en, answer_hi, active, verified	an approved Q&A; fed to the LLM context and matched exactly first

category▼ values: thana, health_centre, vet_camp, parking, ghat, accommodation, toilet_water, lost_found, help_desk. **Adding a new category needs no code:** it shows up in the menu automatically once the category has at least one active, verified row. Its menu label comes from the category_labels rows in settings.

Example rows (from your sample, cleaned):

```
places: TH01 | thana | Nakhas Thana | नखास थाना | ... | Vishal Anand | 9308642005 | Rakesh Kumar |
6207037337 | Srijan Mishra | 8252877610 | | | TRUE | TRUE | Ramesh
places: VC08 | vet_camp | Veterinary Hospital, Sonpur | ... | (s1-s3 blank) | Dr. Vijay Prasad Mandal |
9155688071 | ... | TRUE | TRUE | Ramesh
control: Sanitation & Water | ... | allday_name: Shri Nikhil Kumar, AE PHED Chhapra | allday_phone:
8709898293 | phone_2: 7970663565
```

Spreadsheet 2: "Saran Officer Directory – Bot Data"

Tab	Columns	One row =
README	rules	n/a
officers	id, level▼ (district, subdivision, block, circle, police, other), unit_en, key (TRUE = show on the block summary card), unit_hi, designation_en, designation_hi, short_title_en (≤24 chars), person_name, phone, phone_2, email, office_address, keywords (comma-separated citizen words, e.g. ration, राशन, PDS), sort, active, last_verified, verified, updated_by	one post. Transfers are made by editing the name/phone on the same row , never by adding a row
units	unit_en, unit_hi, level▼, aliases (e.g. Sadar, Chapra, छपरा)	one block, circle, subdivision or thana name, used for search and menus

Why keywords matters: citizens ask by need ("pension", "zameen", "ration card"), not by designation. A data owner who fills keywords improves search more than any code change does. The LLM only falls back to it when keywords miss.

Deliverables for this format (built in M1/M2):

- `data/templates/Sonpur_Mela_Bot_Data.xlsx` and `data/templates/Saran_Officer_Directory_Bot_Data.xlsx`, with dropdowns, plain-text phone columns, frozen headers, the README tab, and your sample already converted in. You upload them to Google Drive as Sheets.
- `scripts/convert_samples.py`, which converts the print-layout files you already have into the template, plus a flag report.
- `docs/data-entry-guide.md` (hi + en, 2 pages) for the data owners.

Validation (run by the sync every 10 min):

- **Errors:** missing required fields, invalid dropdown values, phone rules (D3/D4), Kruti Dev text (D2), duplicate `id`, or a shift name without a phone. **Result:** the tab is not updated, the last good data is kept, and one `admin_alert` per hour lists the tab and row numbers.
- **Warnings:** the same phone on different people (D7), no events in the next 30 days (D11), or `last_verified` older than 60 days. **Result:** they go into the daily digest.
- **Production:** only rows with `active = TRUE` and `verified = TRUE` are published.

6. Service designs

6.1 `svc_mela` (`id_prefix mela`). The menu is built from the data (empty categories are hidden)

Row id	Title (en / hi)	Answer
<code>mela:today</code>	Today's programme / आज का कार्यक्रम	Today's events . If none: "No programme listed today; next: <date> – <programme>". Ends with a [ Full schedule] button (<code>mela:schedule</code>), which frees a list row for  Near me and keeps the menu within the 10-row limit
<code>mela:near</code>	 Near me / मेरे पास	Asks which service (thana / health / vet / parking / ghat / control), then sends a "Send location" request (§6.1a). Shown only once at least one category has coordinates
<code>mela:control</code>	Control room / कंट्रोल रूम	Public helpline number(s) first, then on-duty desk officers for the current shift (Magistrate, Police, Sanitation & Water, Electricity, Health)
<code>mela:cat:thana</code>	Police stations / पुलिस थाना	All 14 temporary thanas with the current-shift in-charge + phone in one card (≈1.2k chars), with a shift label and time
<code>mela:cat:health_centre</code>	Health centres / स्वास्थ्य केंद्र	5 centres + current-shift staff
<code>mela:cat:vet_camp</code>	Veterinary camps / पशु चिकित्सा	11 camps + doctor + phone (no shifts)
<code>mela:cat:parking</code>	Parking / पार्किंग	18 locations (map link where lat/lon exist)
<code>mela:cat:ghat</code>	Ghats / घाट	6 ghats (map link where lat/lon exist)
<code>mela:rules</code>	Do's & don'ts / क्या करें, क्या न करें	Hidden until <code>guidelines</code> has rows
<code>mela:ask</code>	Ask a question / सवाल पूछें	Next free text → LLM Q&A over <code>qa_context</code>

- **Card buttons:** thana, health, and vet cards end with [📍 Nearest] [🔄 All shifts] [🏠 Menu] (the 3-button maximum). `mela:roster:<category>` sends all 3 shifts; `mela:near:<category>` jumps straight to the location request.
- **Long messages:** any card over 4096 chars is split at place boundaries.
- **Free-text shortcuts:** the classifier maps "thana", "police", "doctor", "hospital", "parking", "ghat", "aaj ka program" and similar straight to these ids. A place name typed as text (e.g. "Kali Ghat") matches `places` by trigram and returns that one place's card.

6.1a "📍 Near me": nearest police / health centre / vet camp, with walking directions

Triggers

- Free text such as "police near me", "nearest hospital", "पास में थाना कहाँ है", or "dispensary kidhar hai". The classifier returns `mela.near{category}`.
- The 📍 Near me menu row.
- A card's [📍 Nearest] button.
- A citizen sharing a location with no pending question: the reply covers the nearest **thana + health centre + control room** together.

Flow

1. **Ask for location:** the bot sends `location_request` ("Tap *Send location* to find the nearest police station"). Session state becomes `mela.await_location{category}`, valid for 10 min. [Likely] The WhatsApp Cloud API supports this interactive type; if it is unavailable on the pinned Graph version, the bot falls back to the text instruction "📍 → Location → Send current location".
2. **Receive location:** the citizen's `location` message arrives, and the bot calls `nearest(category, lat, lon, 3)`.
3. **Reply:**
 - One text card with the 3 nearest sites. Each shows name, **≈ distance** (straight-line, labelled "approx."), the **in-charge on duty now** with 📞, and a 🚶 **Walking directions** link: <https://www.google.com/maps/dir/?api=1&destination=<lat>,<lon>&travelmode=walking>.
 - The link deliberately has **no origin**, so Google Maps uses the phone's live GPS and keeps updating as the person walks. It also keeps the citizen's own coordinates out of any URL.
 - A native **location pin** for the nearest site; one tap opens it in the phone's Maps app.
 - Buttons: [🔄 Next 3] [📄 All <category>] [🏠 Menu].

Edge cases

Situation	Behaviour
Citizen is more than <code>mela_radius_km</code> from the Mela centre	"You seem to be outside the Mela area." The bot shows the full category list plus a directions link to the Mela control room
No site in the category has coordinates yet	"Location service for <category> starts soon." The bot shows the full list card (no dead end)
Citizen can't or won't share location (feature phone, GPS off, refuses)	"Or type a nearby landmark (e.g. Kali Ghat, Meena Bazar)." A trigram match on <code>places</code> uses that place's coordinates as the origin
Location shared while the session expected something else	Handled as "nearest of all key categories"
Two sites at about the same distance	Both are shown; the list is sorted by distance, then <code>sort</code>

Privacy:

- The citizen's latitude/longitude are **used in memory only and never stored**.
- `message_log` records `kind=location`, the category, and a distance bucket (<250 m, <500 m, <1 km, <2 km, more) for analytics.
- The privacy notice mentions location use.

6.1b Admin coordinate capture (on-site, in the set-up week)

Typing coordinates by hand into a sheet is error-prone (swapped lat/lon, missing decimals), and the temporary sites only exist once they are built. Staff therefore **pin each site by standing at it and sharing their WhatsApp location with the bot.**

1. An admin number sends `pin` (or taps `mela:adm:pin`), picks a category, then the site. The list shows sites **without coordinates first**, marked /, and is paginated.
2. The bot sends `location_request`; the admin shares the current location from the site's entrance.
3. The bot replies with the captured point as a map pin, plus  Save] [ Retry].
4. On Save:
 - the point is upserted into `place_coords`
 - it is written back to the sheet's `lat/lon` cells as best effort, which needs the service account to have Editor access on `places`
 - the event is logged with the admin's hash
5. `pin status` shows counts per category (e.g. "Thana 11/14 pinned"). The daily digest lists unpinned sites until they are all done.

Effort on the ground: about 50 sites. One or two staff on a bike can do it in about a day [Guessing], once the camps and thanas are physically up, typically in the week before the Mela opens.

6.2 `svc_directory` (`id_prefix dir`). Navigation by office level, matching the data (D12)

Level menu (list):

- `dir:lvl:district`: District officers (33)
- `dir:lvl:subdivision`: Sub-division officers (SDO/DCLR, 6)
- `dir:lvl:block`: Block officers (BDO, CO, and all other block-level posts)
- `dir:lvl:police`: Police stations (39)
- `dir:search`:  Type a name/post

Block (`dir:blk:p1, p2`; 20 blocks shown as 9 per page + "More ►"):

- Tapping a block (`dir:blk:<name>`) returns a **key-officers card** (BDO + CO + the posts marked `key=TRUE`, e.g. MOIC, BEO, CDPO), since citizens rarely know which one they need. It ends with  All officers of `<block>`].
- `dir:blkall:<block>;p1` shows every block-level post as a paginated list: BAO/BHO, BEO, MOIC, CDPO, BCO, MO (Supply), JE/AE, Panchayat Raj Officer, Labour Enforcement Officer, BWO, BPM (JEEViKA), RO, and others. Tap → the officer card.
- **Search is the main path for these posts.** "Dighwara ka hospital doctor" → MOIC Dighwara. "anganwadi Sonpur" → CDPO Sonpur. It works through `keywords` plus the LLM need → designation mapping.

District (33 posts):

- A paginated list of designations. Titles are shortened by a `short_en` rule (e.g. "District Supply Officer" → "Supply Officer").
- Tap → officer card.

Sub-division: Sadar / Marhaura / Sonpur → a card with the SDO + DCLR.

Police (39 stations): a paginated list (5 pages), or type the thana name (trigram).

Search / free text, e.g. "BDO Dighwara", "ration card officer", "DM ka number":

1. Trigram match on unit, designation, and name (+ aliases).
2. If there is no confident match, the LLM maps the need to a designation from the 33+ known posts via the `keywords` column. For example, ration card → District Supply Officer; land → CO/DCLR; pension → ADSS Social

Security.

3. The bot shows the top 1 match, or up to 3 as buttons.

Officer card: designation, name (suffix stripped), 📞 phone, ✉ email (omitted if blank), unit, "Verified: <date>".

7. Security & privacy

- **Network:** there are no inbound ports on the office machine. The Cloudflare Tunnel exposes only `/webhook/*` and `/healthz`. The n8n editor, Postgres, and Metabase are bound to 127.0.0.1 and reached over Tailscale.
- **Secrets:** kept in `.env` (chmod 600), never committed. Includes `N8N_ENCRYPTION_KEY`, `META_APP_SECRET`, `PHONE_HASH_SECRET`, and `PGCRYPTO_KEY`, all backed up offline.
- **Webhook authenticity:** the HMAC signature check (E2), plus the verify-token handshake.
- **DPDP Act 2023:**
 - **Notice:** a one-line privacy notice on first contact.
 - **Minimisation:** analytics use the hash only; the raw number is encrypted at rest.
 - **Retention:** 180 days, enforced by the nightly purge.
 - **LLM:** receives message text only.
 - **Location:** citizen coordinates are processed in memory and never stored or logged; only distance buckets are logged. Admin captures store the site's coordinates, not the person's.
- **Ethics:** the bot never asks for Aadhaar or other identity numbers.

8. Operations

- **Deploy:** `git pull && scripts/n8n_import.sh && psql -f sql/*.sql`. Credentials are created once per environment and documented.
- **Backups:**
 - nightly `pg_dump` of both DBs, 14-day local rotation plus an off-box copy
 - weekly copy of the env files and encryption keys to secure storage
 - one restore drill before go-live
- **Monitoring:**
 - an external uptime check on `/healthz` every 5 min
 - the Error Trigger → `admin_alert`
 - a sync-staleness check: no OK sync in 1h → alert
 - the daily report includes error count and p95 latency
- **docs/runbook.md:** covers restart, token rotation, "wrong number published" hotfix (edit sheet → sync or force-run), disabling a service, disabling the LLM, and rollback (re-import the previous git tag).

9. Testing strategy

Layer	Tool	What it covers
SQL	<code>tests/sql/*.sql</code> via <code>psql</code> + <code>pgTAP</code> -style asserts	<code>begin_turn</code> (dedupe, rate limit), <code>current_shift / on_duty</code> shift boundaries, <code>nearest()</code> (known points with hand-computed distances, inactive/unverified/no-coordinate sites excluded, sheet value overrides capture), <code>replace_*</code> atomicity (sync leaves <code>place_coords</code> intact), directory fuzzy block match
Templates	<code>tests/check_templates.py</code>	WhatsApp field limits in hi and en, and no missing translations

Layer	Tool	What it covers
Flow	tests/run_flow_tests.py + tests/payloads/*.json + tests/services/<key>/*.yaml	Real-shaped Meta payloads are sent to the staging webhook (mocks on). Asserts on captured outbound payloads, <code>message_log</code> , and <code>sessions</code>
Contract	same runner via 99-test-harness	Output schema of every service for its standard inputs
LLM routing	tests/llm/free_text.yaml (50 hi/Hinglish/en lines, labelled)	Run against the real API; ≥90% correct service/subtype
Load	tests/k6/load.js	30 msg/s for 10 min against mocks: p95 <3s, 0 lost, 0 duplicates
UAT	docs/uat-mX.md	Real phones, every path, both languages

CI: GitHub Actions `ci.yaml` runs on every push:

1. Starts compose in `ENV=test` with the mocks.
2. Applies the SQL.
3. Imports the workflows.
4. Runs the SQL tests, template checks, and flow and contract tests.

The LLM-routing and load tests are run manually at their gates.

10. Milestones, tasks and estimates

Effort is in developer-days for 1 developer, plus a part-time data/ops owner on the client side.

Week-1 prerequisites (client side; these block go-live)

- ☒ WhatsApp number live and verified
- ☐ Meta: share the permanent System User token, Phone Number ID, **App Secret** (for signature checks); submit the `admin_alert` utility template; keep a separate **Meta test number for staging**
- ☐ Office machine meeting §2.2 (dedicated, Ubuntu-ready), UPS, 4G failover
- ☐ A (sub)domain with its DNS on Cloudflare (free account); IT confirms outbound HTTPS to Cloudflare, Meta, Anthropic and Google is allowed
- ☐ Off-site backup target (a Google Drive folder or similar)
- ☐ Written approval for the hosted LLM (otherwise go live with `LLM_ENABLED=false`)
- ☐ Google service account; Mela and Directory sheets created and shared
- ☐ Named data owners for the Mela sheet and the Directory sheet
- ☒ Sample data received: `Sonpur_Mela_Data.xlsx` (7 tabs) and the Directory Google Sheet (118 officers)
- ☐ **Public control room helpline number(s)** for the Mela (D8)
- ☒ Publishing duty-staff mobiles approved (D15): file a copy of the order
- ☐ Unicode spelling of "Nakhas" (the cell uses Kruti Dev encoding, D2); corrected numbers for D3/D4; confirm the D7 duplicates
- ☐ **Block-level officer lists for all 20 blocks** (BDO/CO plus the other posts: MOIC, BEO, CDPO, BAO, MO Supply, JE, and so on), in the template format or any Excel
- ☐ **Mela centre point and radius** (e.g. the Harihar Nath temple gate, 5 km) for the "outside the Mela area" check

- ☐ **Field staff (1–2 people, about 1 day) in the Mela set-up week** to pin every thana, health centre, vet camp, parking area and ghat with the admin tool (§6.1b); their numbers are added to `ADMIN_WA_NUMBERS`
- ☐ Service account given **Editor** access to the Mela sheet (for coordinate write-back)
- ☐ Optional: accommodation list, do's & don'ts, Hindi names
- ☐ The 2026 programme schedule, when announced (the sample is 2025)

M0: Core platform: weeks 1–2, ~10 dev-days

Task	Days
Office machine setup: Ubuntu, Docker, systemd, Tailscale, cloudflared tunnel + access rules, UPS/boot settings	1
Compose rewrite: pinned versions, Postgres dbs, env, fixes E4–E6; staging + prod stacks	1
<code>00_core.sql</code> : tables, <code>begin_turn</code> / <code>end_turn</code> , views, seed services/templates	1.5
Router <code>00-router</code> : signature check, explode, dedupe, normalize, route, dispatch, end turn	2
<code>01-send</code> : renderer, limit validator, phone guard, retries, logging	1
<code>02-llm</code> : classifier + kill switch; <code>08-error</code> ; <code>03-admin-alert</code>	1
<code>svc_template</code> + <code>svc_echo</code> ; <code>99-test-harness</code>	0.5
Mocks, flow runner, template checker, CI	2
<code>n8n_export.sh</code> / <code>n8n_import.sh</code> , fixed IDs, runbook skeleton	1

Gate M0:

- CI green.
- Flow tests pass for: text, button, list, location, voice, status callback (no execution saved), batched webhook (both messages answered), duplicate ID (one reply), bad signature (rejected), stale button (correct), double-tap (consistent), rate limit.
- `svc_echo` on/off changes the menu, and a 4th registry row flips the menu to a list.
- A fresh-instance import works with credentials as the only manual step.
- A real round-trip works on the Meta test number.

M1: Sonpur Mela: weeks 3–4.5, ~12.5 dev-days

Task	Days
<code>10_svc_mela.sql</code> : shifts, places, duty, control room, events, <code>current_shift</code> / <code>on_duty</code> , <code>replace_*</code>	1
Template workbooks (dropdowns, text phone columns, README) + <code>scripts/convert_samples.py</code> (print layout → template: expand merged cells, split multi-phone cells, detect Kruti Dev text, flag bad phones) + conversion report + data-entry guide	2
<code>sync_mela</code> with validation (D2–D7, D11) and alerts/digest	1.5
<code>svc_mela</code> workflow: data-driven menu, today/schedule, control room on-duty, category cards with current shift, all-shifts roster, place-name lookup, ask (Q&A mode)	3
Fixtures (built from the real 2025 sample, re-dated) + SQL shift tests + 30-line LLM set	1.5
📍 Near me (§6.1a) : <code>nearest()</code> SQL, location request/receive, radius check, landmark fallback, directions link + map pin, card buttons; core send support for <code>location</code> / <code>location_request</code> messages	2
Admin coordinate capture (§6.1b) : <code>core.admins</code> , <code>mela:adm:*</code> flow, <code>place_coords</code> , sheet write-back, <code>pin_status</code> , digest line	1
UAT on staging with the data owner (including a walk test at 3 real points with the admin tool and near-me)	0.5

Gate M1:

- All fixtures pass in hi and en.
- Shift tests pass: 05:59/06:00, 13:59/14:00, 21:59/22:00, and 02:00 (Shift-3 across midnight). Merged-cell desks show on every shift.
- The converter reproduces every source row: 5 health centres × 3 shifts, 14 thanas × 3, 11 vet camps, 18 parking, 6 ghats, 14 events, and 5 control desks.
- The converter report flags D2, D3, D4 and D7 items, and none of them reaches citizens.
- A broken sheet leaves the data unchanged and raises one alert.
- LLM routing ≥90%, with 0 invented numbers; the 30-line set includes 8 "near me" phrasings in Hindi, Hinglish and English.
- Near me: for 10 test origins, the returned top-1 matches a hand-checked nearest site. Directions links open walking navigation to the right point on Android and iPhone. Origins outside the radius, no-coordinate categories, and the landmark fallback behave as §6.1a says.
- Admin capture: a non-admin gets "not allowed" for `me!a:adm:*`; a capture survives the next 10-min sync; a sheet-typed value overrides it.
- No citizen latitude/longitude appears in `message_log`, the n8n execution data, or the DB (grep check).
- The data owner signs off that the staging answers match the sheet.

M2: Directory + go-live: weeks 5–6, ~8 dev-days

Task	Days
<code>20_svc_directory.sql</code> + blocks aliases; convert the 5 stacked sections → officers (118 rows, sub-division from email); <code>sync_directory</code>	1.5
<code>svc_directory</code> : level menu, block key-officers card + all-posts pagination, district/sub-division/police lists, trigram search, LLM need → designation mapping, card	3
Fixtures; confirm no core feature change	1
k6 load test (+ queue mode only if it fails)	1
Backups off-site + restore-to-cloud drill (§2.3), UptimeRobot, power-cut/ISP-failover test, runbook complete	1.5
Prod stack + production number cut-over, UAT with about 20 staff phones, Hindi copy review	1

Gate M2:

- Directory fixtures pass, with no core feature change.
- 30 real-style queries resolve to the right officer in ≥90% of cases, e.g. "BDO Dighwara", "Sonpur CO", "ration card complaint", "DM number", "Derni thana".
- Load test meets the targets.
- The restore-to-cloud drill completes in under 1 hour.
- Pulling the power cable and the primary internet cable: the bot recovers without anyone touching it.
- UAT sign-off.
- **Go-live around 10 Nov**, with the prod webhook switched to the production number.

M3: Analytics & feedback: weeks 7–8 (during the Mela), ~5 dev-days

Task	Days
Metabase container + read-only DB user on views	1
Dashboards: daily users/messages, per service/subtype, top free-text themes, 🗨️ rate, unanswered list, peak hours, LLM tokens/cost	2
85-daily-report at 08:00 IST via <code>admin_alert</code> + email; 09-purge job	1.5
Weekly unanswered review → keyword/menu/data fixes	0.5 (+ ongoing)

Gate M3:

- A sampled day reconciles with `message_log`.
- The purge is verified on a copy.
- No raw numbers are visible anywhere in Metabase.

Phase 1 total: about 37.5 developer-days. The go-live path (M0–M2) is about 32.5 days (≈6.5 weeks), which leaves **only 2–3 days of buffer** before ~10 Nov.

Slip rule: if M1 runs late, 📍 Near me and admin capture are the first items to move. They can ship during the first Mela week without affecting anything else, because the feature stays switched off until coordinates exist anyway. Coordinates are captured in the set-up week, so the feature realistically goes live **when the camps are up (around 17–23 Nov)** whichever way the schedule falls.

11. Risk register

Risk	Likelihood	Impact	Mitigation
admin_alert template not approved	Low	No WhatsApp staff alerts	Email/SMS alerts as backup
LLM approval refused	Medium	Weaker free-text handling	<code>LLM_ENABLED=false</code> still covers every service via menus
Office machine / power / internet outage during the Mela	Medium–High	Bot silent	\$2.2 hardening, UptimeRobot SMS alerts, 1-hour cloud move (\$2.3), rehearsed
Office IT blocks Cloudflare/Anthropic egress	Medium	Can't go live on-prem	Confirm in week 1; fallback to the Caddy + static IP profile, or the cloud VPS
Mela data not verified in time	High	Wrong numbers published	Named owner, <code>verified</code> column, prod rejects unverified rows, sign-off in the M1 gate
Officer transfers make the directory stale	High	Wrong contact	<code>last_verified</code> shown on every card; monthly verification reminder to the owner
Traffic spike during the Mela	Low–Medium	Slow replies	Load test on the actual office machine; queue-mode profile ready
Site coordinates not captured, or captured wrongly	Medium	Near me unavailable, or it points to the wrong spot	Admin tool with map-pin confirmation; <code>pin_status</code> in the daily digest; the feature hides categories without coordinates; a sheet value can correct any point
Citizens' GPS inaccurate in dense crowds	Medium	"Nearest" off by 100–200 m	Show the top 3 with "approx." distances; the walking link recalculates live in Maps
Citizens send voice notes	High	Unanswered	Friendly redirect to the menu; log volume; voice-to-text as a future service
n8n upgrade breaks nodes	Low	Outage	Pinned versions; upgrade only through staging + CI

12. Running cost estimate (Mela month) [Guessing: volumes are assumptions]

- **Hosting:** office machine (existing or about ₹60–80k one-time if bought), UPS, and a 4G backup plan at about ₹300–500/month. Cloudflare Tunnel is free. The domain costs about ₹800/year.

- **WhatsApp:** [Likely] free for replies to citizen-initiated chats (the service window). Utility templates for admin alerts cost a few paise to about ₹0.15 each.
- **Claude Haiku 4.5** (\$1 input / \$5 output per MTok):
 - Assume 50k free-text calls × ~1.5k input + 200 output tokens.
 - That gives ~ \$75 input + \$50 output, about **\$125 (~₹10.5k)** for the whole Mela.
 - Button taps cost nothing because they never call the LLM.

13. Immediate next steps

1. **District / client side:** close the week-1 prerequisites in §10. The ones that most affect the schedule are:
 - the office machine and the Cloudflare domain
 - the Meta App Secret and a test number
 - the public control room number
 - the block-level officer lists
 2. **Development:** start M0 (core platform) on the staging stack as soon as the office machine is available. Until then, development runs on a laptop with the same Docker setup.
 3. **Data owners:** receive the template workbooks and the data-entry guide in M1, and start moving the 2026 duty orders into them as soon as the orders are issued.
-

14. Later phases (out of Phase 1 scope)

How any new service plugs in (no router change)

1. Copy `svc_template` and give it a new fixed workflow ID.
2. Add a new `svc_<key>` DB schema.
3. Write fixtures and pass the contract test.
4. Insert one `core.services` row and its templates.
5. Enable it on staging, run UAT, then enable it on prod.

The main menu grows automatically. In Phase 1 it shows **2 reply buttons**; with more than 3 services it switches to a list.

Scheme Eligibility (next phase): notes kept for later

- **Data source:** structured MySQL columns, exposed through a read-only view and synced nightly into its own `svc_schemes` schema.
- **Verdict:** decided by a deterministic SQL function, never by the LLM.
- **Flow:** about six button/list questions, each with a skip option.
- **Output:** each result gives the reason and an "indicative only" disclaimer.
- **Effort:** about 10 dev-days.
- **Needs:** a MySQL schema sample and department-reviewed test profiles.